

C2HLSC: Can LLMs Bridge the Software-to-Hardware Design Gap?

Luca Collini *Graduate Student Member, IEEE*, Siddharth Garg *Member, IEEE*, Ramesh Karri *Fellow, IEEE*

Abstract—High Level Synthesis (HLS) tools offer rapid hardware design from C code, but their compatibility is limited by code constructs. This paper investigates Large Language Models (LLMs) for refactoring C code into HLS-compatible formats. We present several case studies by using an LLM to rewrite C code for NIST 800-22 randomness tests, a QuickSort algorithm and AES-128 into HLS-synthesizable c. The LLM iteratively transforms the C code guided by user prompts, implementing functions like streaming data and hardware-specific signals. This evaluation demonstrates the LLM’s potential to assist hardware design refactoring regular C code into HLS synthesizable C code.

Index Terms—Chip Design, LLM, Catapult HLS, Cryptocores.

I. INTRODUCTION

High-Level Synthesis (HLS) is a promising chip design methodology that enables rapid hardware design from high-level specifications. HLS tools convert a high-level specification (C, C++) into an register transfer level (RTL) description [1]: (1) HLS uses state-of-the-art compilers (e.g., LLVM or GCC) to extract a high-level control data flow graph (CDFG). (2) They then assign operations to time (scheduling) and space (allocation and binding) to determine the micro-architecture. HLS tools also support pragmas and directives to explore architectural choices for a C specification. Software and hardware paradigms are different. Hence HLS tools support a subset of C constructs that map to hardware. For instance, hardware does not support dynamic memory allocation and recursive constructs. Outputs can only communicate through parameters, arrays sizes need to be static, limit support for pointers and multiple processes can be modeled through independent function instances mapped into hardware blocks. Designers manually refactor C code to remove these constructs and make it compatible with HLS tools. However, manual refactoring is time consuming, and error prone [2].

A. Large Language Models (LLMs)

LLMs are trained on massive amounts of text data and excel at tasks like code generation and translation, particularly in languages like C, C++, and Python. However, their performance suffers on Hardware Description Languages (HDLs) like Verilog or VHDL due to the limited amount of training data available in those languages [3]. This paper explores LLMs for refactoring C code, transforming it into a subset C compatible with HLS tools. LLMs can analyze generic C code and refactor it to HLS-synthesizable C. This leverages the power of LLMs in generating and manipulating C code and bypassing their limitations in generating HDL code.

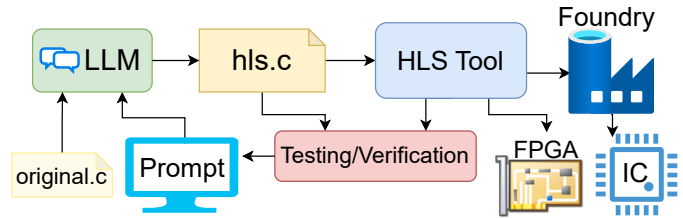


Fig. 1: Flow for the proposed C2HLSC approach.

B. Contributions and Roadmap

LLMs have limited success in generating Verilog code [4], [5]. This is due to the scarcity online of hardware description language (HDL) data compared to code in C, C++ and other software. We explore use of LLMs to assist developers in refactoring generic C specification into synthesizable C that is compatible with HLS as shown in Figure 1. Our work has two main contributions. First, we present case studies of deriving HLS-usable C codes derived from generic C codes. The study compares development time and resource use achieved using LLM-assisted development in lieu of cumbersome manual methods. Second, we discuss the results and present a prototype hands-free C2HLSC LLM-based tool, which will be open-source. The paper road-map is as follows: 1) Section II presents related work, highlighting the novelties of the approach. 2) Section III illustrates the case study, presenting the tasks and methodologies and results. 3) Section IV presents the prototype hands-free C2HLSC tool, discussing capabilities and limitations. 4) Section V concludes the paper.

II. RELATED WORK

Previous work explored LLMs to design hardware. Verigen fine-tuned an LLM to improve its ability to produce Verilog [4]. The fine tuned LLM though performed marginally better than ChatGPT3.5-turbo with an accuracy ~65%. ChipChat [5] was the first to tapeout a design written by an AI model. However, the single shot performance of the AI model was low and needed several iterations in order for the LLM to get to the correct result. We target generating synthesizable C code as LLMs are more capable at C than at hardware languages [3]. In [6] an LLM was used to write Amaranth HDL, a Python based HDL, that allows to model synchronous logic at the RTL. For this reason, while it uses a high level language, its semantics are close to verilog, and targets hardware designers. While the LLM came up with parts of the design, it fell short in some tasks, like generating

interfaces. Software developers use HLS to design hardware and as such the code only provides the functionality¹.

III. CASE STUDY

A. Overview

We evaluated Gemini LLM [7] to transform C code into synthesizable C suitable for HLS. The evaluation consisted of two tasks. The **first task** involved rewriting reference C implementations of the Frequency test, Frequency Block test, Cumulative Sums, and Overlapping Template Matching tests from the NIST 800-22 suite [8] into synthesizable C code. These tests are designed to assess the randomness of a sequence. A **first challenge** arose due to the inherent differences between software and hardware implementations. The reference C implementations operate on a pre-loaded random sequence stored in memory. Conversely, hardware implementations require on-the-fly analysis, processing the sequence bit-by-bit. This necessitates modifying the code to handle a streaming data input rather than a pre-loaded array. A **second challenge** stemmed from the p-value calculation. In software context, the precise p-value is critical and computed on-the-fly. However, since the hardware implementations primarily focus on distinguishing random from non-random sequences one can simplify this by pre-computing certain values offline and reducing the computational burden during on-the-fly analysis. Both these challenges are non-trivial for human developers and LLMs. The **second task** assesses the LLM's ability to rewrite code constructs that are not supported by HLS tools. We used two algorithms: a QuickSort containing pointers and recursion [9], and the AES128 encrypt from the tinyAES library [10] with six functions. The goal was for the LLM to generate code without pointers and recursion, making it suitable for (Catapult) HLS.

B. Methodology

We broke down the process into small steps to allow the LLM to transform the original C into synthesizable C. For the first task we followed the following steps for the three tests: 1) Present task to the LLM: "Hi, I have this code in C that I need to rewrite such that I can use it with an HLS tool to generate hardware.". 2) Ask to remove print statements. 3) Ask to rewrite the function as a streaming interface: "Now I need to rewrite the function such that it will get inferred as a streaming interface, to do so, I need to get rid of the epsilon array and have the function take a parameter to accept a bit at each function call." 4) Ask to remove math steps to be computed offline (in some cases, ask to write a script to run them). 5) Ask to add *is_random* and *valid* signals as parameters. 6) Ask to optimize data types using arbitrary width integers and fixed point arithmetic using HLSLIBS [11]. 7) Ask to write a main function to test the function passing random bits. 8) Ask to fix mistakes passing errors from HLS tool.

For QuickSort we followed these steps: 1) Present the task to the LLM: "Hi, I have this code in C that I need to rewrite

such that I can use it with an HLS tool to generate hardware.". 2) Ask to remove print statements. 3) Ask to rewrite function without using pointers. 4) Ask to rewrite function without recursion. 5) Ask to fix array sizes in function parameters. 6) Ask to optimize data types using arbitrary width integers and fixed point arithmetic using HLSLIBS. 7) Ask to write a main function to test the function passing an array to sort. 8) Ask to fix mistakes by passing errors from HLS tool.

For the AES 128 from tinyAES [10] we followed the following steps asking to fix one function at a time: 1) Present the task to the LLM: "Hi, I have this code in C that I need to rewrite such that I can use it with an HLS tool to generate hardware.". 2) Ask to rewrite for loops with fixed bounds and no pointer usage. 3) Ask to rewrite the function parameters to using fixed size arrays. 4) Ask to fix eventual mistakes passing errors from the HLS tool. When the LLM responds with sub optimal answers, we check alternative answers, and if none fully satisfied the request we instruct the LLM with additional prompts including more details pointing out where the problem was, and, if not sufficient, hinting at possible solutions.

C. Results

The aim of this study is to evaluate how LLMs perform at rewriting C code such that it is HLS synthesizable². We run the code through Catapult HLS to check correctness after synthesis, but we do not focus on the resource utilization, as it depends on the architectural decisions. We targeted the nangate45 library at 50 MHz with a synchronous active high reset for all the tests. The LLM was able to rewrite all C codes to run on Catapult HLS. We performed simulations with Modelsim to check result equivalency between the original C and the synthesized Verilog obtained from the LLM-generated C. The original C codes (without printf) and the HLS C code rewritten by the LLM are in Appendix A, links to the conversations are in Appendix C. We can classify the errors in the LLM generated code into compile/synthesis errors, and functional errors. The former were easier to fix instructing the LLM with the error message and affected line. For functional errors caught by the failing tests, it was harder for the LLM to identify the cause and propose a fix. Manual effort was needed to point out sources of the issues for the LLM to propose an effective fix. Typical functional issues where mistaken loop readjustments, widths and arbitrary widths types.

1) *C2HLS C of NIST 800-22 Randomness Tests:* The LLM performed well in most steps, especially in rewriting the function as a streaming interface, which is one of the more complex steps. Figure 2 shows the streaming interface rewriting for the Frequency Block test. For all the three tests this step was performed correctly at the first shot. However, the LLM did stumble on tasks that seemed more trivial. For example, when asked to remove the *abs* function in this line of code (`abs(sum.to_int()) <= SUM_THRESHOLD`);

¹Whereas the hardware architecture and interface specification are instructed using HLS pragmas and directives.

²Finding pragmas to optimize the hardware architecture is an orthogonal problem for which LLMs could be employed, left as future work.

```

1 I need to rewrite the function such that it
2 will get inferred as a streaming interface,
3 to do so, I need to get rid of the epsilon
4 array and have the function take a parameter
5 to accept a bit at each function call.

1 void BlockFrequency(int M, int n) {
2   int i, j, N, blockSum;
3   double p_value, sum, pi, v, chi2;
4   N = n / M; /* # OF SUBSTRING BLOCKS */
5   sum = 0.0;
6   for (i = 0; i < N; i++) {
7     blockSum = 0;
8     for (j = 0; j < M; j++) {
9       blockSum += epsilon[j + i * M];
10    }
11    pi = (double)blockSum / (double)M;
12    v = pi - 0.5;
13    sum += v * v;
14  }
15  chi2 = 4.0 * M * sum;
16  p_value = cephes_igamc(N / 2.0, chi2 / 2.0);}

1 Here's the code rewritten to accept a single
2 bit per function call, making it more suitable
3 for a streaming interface:

1 void BlockFrequency(int M, int n, bool bit) {
2   static int i = 0, j = 0, blockSum = 0;
3   static double sum = 0.0;
4   double pi, v, chi2;
5   if (j == 0) { // Reset on new block
6     i++;
7     blockSum = 0;
8   }
9   blockSum += bit; // Accumulate bit
10  j++;
11  if (j == M) { // Block complete
12    pi = (double)blockSum / (double)M;
13    v = pi - 0.5;
14    sum += v * v;
15    j = 0;
16  }
17  if (i == (n / M)) { // All blocks processed
18    chi2 = 4.0 * M * sum;
19    p_value = cephes_igamc(n / 2.0, chi2 / 2.0);
20  }}

```

Fig. 2: Transforming into a streaming pipeline.

it came up with convoluted ways that did not work like `(sum.to_int() < 0) ? -sum.to_int() : sum.to_int()) <= SUM_THRESHOLD` and it took many tries to eventually get the expected result `(sum < SUM_THRESHOLD) && (sum < -SUM_THRESHOLD)`. The test function generation was always generated effectively.

2) *C2HLSC of QuickSort*: The original QuickSort used a function to swap elements in the arrays using pointers. Inlining the function and removing pointers was flawless as reported in Figure 3. However, in the subsequent steps it reverted to using the function and needed re-prompting to inline. The LLM got rid of recursion in one shot, using a fixed size stack to track the high and low values. However, the stack management logic was not correct. The problem was identified at the end, when running the generated test, which caused a segmentation fault. The LLM fixed the problem in a few tries by feeding the errors

```

1 Can we get rid of the swap function and replace
2 it inline?

1 void swap(int* a, int* b) {
2   int temp = *a;
3   *a = *b;
4   *b = temp;}
5 ...
6   if (i < j) {
7     swap(arr[i], arr[j]); // Call to
8       modified swap function
9   }
10 swap(&arr[low], &arr[j]);
11 ...

1 Yes, you can definitely get rid of the swap
2 function and replace it inline within the
3 partition function. Here's the modified code:

1 ...
2   if (i < j) {
3     // Inline swap using temporary variable
4     int temp = arr[i];
5     arr[i] = arr[j];
6     arr[j] = temp;
7   }
8   // Swap pivot element with element at index j
9   int temp = arr[low];
10  arr[low] = arr[j];
11  arr[j] = temp;
12  return j;
13 ...

```

Fig. 3: Removing pointers and inlining.

from gdb and showing intermediate values of high and low.

3) *C2HLSC of AES 128 Encrypt*: We fed one function at a time to the LLM starting from the top function Cipher. We tested the generated C code and found errors. We prompted the LLM to correct the errors. The LLM made major changes in the MixColumn and ShiftRows functions, so we asked it to double check that the new code was correct. ShiftRow function was corrected in 2 iterations, without extra context on the error. Results are in Figure 4. Fixing MixColumn function took more iterations even though the error was simple.

D. Discussion

Using LLMs for refactoring C code into HLS-compatible formats is a promising avenue in LLM aided design. The LLM, in our case Google's Gemini, refactored reference C implementations of NIST 800-22 randomness tests, QuickSort, and AES-128. The LLM's ability to iteratively refactor the code based on user instructions significantly reduces manual effort in adapting code for HLS tools. This can streamline the hardware design process, especially for repetitive tasks. The LLM effectively addressed challenges like converting code from memory-based data processing to streaming, from recursion to iteration and pointers. While the LLM achieved core functionalities, it occasionally struggled with minor details requiring several iterations to guide it to the correct solution. In a practical scenario, a developer can rectify these minor errors. However, for an automated flow, a feedback loop is crucial, like that in [5].

```

1 Can you fix possible problems in this function?
2
3 static void ShiftRows(state_t* state){
4     uint8_t temp;
5     // Rotate first row 1 columns to left
6     temp = (*state)[0][1];
7     (*state)[0][1] = (*state)[1][1];
8     (*state)[1][1] = (*state)[2][1];
9     (*state)[2][1] = (*state)[3][1];
10    (*state)[3][1] = temp;
11    // Rotate second row 2 columns to left
12    temp = (*state)[0][2];
13    (*state)[0][2] = (*state)[2][2];
14    (*state)[2][2] = temp;
15    temp = (*state)[1][2];
16    (*state)[1][2] = (*state)[3][2];
17    (*state)[3][2] = temp;
18    // Rotate third row 3 columns to left
19    temp = (*state)[0][3];
20    (*state)[0][3] = (*state)[3][3];
21    (*state)[3][3] = (*state)[2][3];
22    (*state)[2][3] = (*state)[1][3];
23    (*state)[1][3] = temp;}
24 ...

```

LLM result after 2 tries telling the LLM that the loop logic was wrong.

```

1 static void ShiftRows(state_t state[4][4]) {
2     uint8_t temp;
3     int i, j;
4     // Loop over rows 1 to 3 (0-based indexing)
5     for (i = 1; i < 4; ++i) {
6         // Number of positions to left shift for
           current row
7         int shift = i;
8         // Use a separate loop for the number of
           shifts
9         for (j = 0; j < shift; ++j) {
10            temp = state[0][i]; // Store the element
              to be shifted
11            // Shift all elements by one position to
              the left
12            for (int k = 0; k < 3; ++k) {
13                state[k][i] = state[k + 1][i];
14            }
15            state[3][i] = temp; // Place the temp value
              at the end
16        }}}

```

Fig. 4: Removing pointers and inlining.

TABLE I: Resource Utilization and Latency Results

Design	Area Score		# Operations		Latency	
	This work	Manual	This work	Manual	This work	Manual
NIST-Monobit	244	225.3	19	19	1	1
NIST-Monobit Block	702.3	826.0	24	20	1	1
NIST-Cusums	677.4	632	24	28	1	1
NIST-Overlapping	9933.4	7172.1	165	118	1	1
QuickSort	18115.8	n.a.	67	n.a.	18	n.a.
AES	38604.5	n.a.	1924	n.a.	160	n.a.

Table I shows area for the implemented designs. For NIST test implementation we have reference designs that were implemented by a graduate student. We used the same directives for a fair comparison between the 2. Area scores from Catapult are close. The manual implementations took around 4 hours each while C2HLSLC took between 30 to 60 minutes each. Although the sample size is limited, this shows the potential of LLMs in speeding up the process effectively and efficiently.

IV. HANDS-FREE C2HSLC PROTOTYPE

With the experience of the engineer-in-the-loop case studies, we implemented a hands-free C2HSLC prototype. The first insight from the case study was the twofold nature of the errors that can occur in the generated C — functional/compile errors and synthesis errors. The former kind can be caught by gcc. The latter kind can be caught by running the Catapult HLS tool. For this reason we setup a double feedback loop as shown in Figure 5. One checks that the generated code compiles and passes reference tests, and one that checks that the code is synthesizable by Catapult HLS. We selected ChatGPT 3.5 Turbo as we did not have access to Gemini APIs. The flow is implemented in Python and will be made open source.

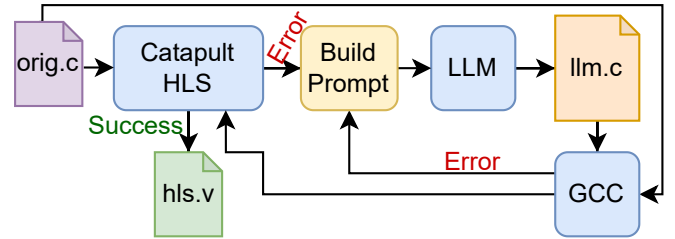


Fig. 5: Hands-free C2HSLC LLM-based prototype flow.

We ran the flow on the QuickSort and AES C codes used in the case study. In the current version, the flow can handle only a limited number of functions at a time and so for AES we broke the code down into the individual functions.

TABLE II: Hands-free C2HSLC LLM-based flow.

Design	# Prompts	Area Score	# Ops	Latency
QuickSort	6	52184	1961	8
AddRoundKey	9	702.3	837	210
ShiftRows	1	23.9	3	0
MixColumns	1	635.9	225	6
SubBytes	1	1176.5	53	33

The flow was able to automatically rewrite the functions without any human intervention. We plan to expand it by allowing more functions at a time, and include better optimizations. The generated codes are in Appendix A.

V. CONCLUSIONS

The case studies demonstrated the potential of LLMs in aiding HLS. The implemented prototype shows the potential of LLMs in aiding hardware design leveraging HLS. Future work includes the use of LLMs to discover architectural trade-offs by applying pragmas and directives. Together with the proposed flow, it will enable a seamless transition from C to HLS C to optimized hardware architectures.

REFERENCES

- [1] R. Nane, V.-M. Sima, C. Pilato, J. Choi, B. Fort, A. Canis, Y. T. Chen, H. Hsiao, S. Brown, F. Ferrandi, J. Anderson, and K. Bertels, "A survey and evaluation of fpga high-level synthesis tools," *IEEE Transactions on CAD*, vol. 35, no. 10, pp. 1591–1604, 2016.
- [2] D. Soni, M. Nabeel, K. Basu, and R. Karri, "Power, area, speed, and security (pass) trade-offs of nist pqc signature candidates using a c to asic design flow," in *IEEE International Conference on Computer Design*, 2019, pp. 337–340.
- [3] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of github copilot's code contributions," in *IEEE Symposium on Security and Privacy*, 2022, pp. 754–768.
- [4] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "Verigen: A large language model for verilog code generation," *ACM Trans. Des. Autom. Electron. Syst.*, feb 2024, just Accepted. [Online]. Available: <https://doi.org/10.1145/3643681>
- [5] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-chat: Challenges and opportunities in conversational hardware design," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, 2023, pp. 1–6.
- [6] J. T. Meech, "Leveraging high-level synthesis and large language models to generate, simulate, and deploy a uniform random number generator hardware design," 2024.
- [7] "Bard: A large language model from google ai," <https://blog.google/technology/ai/bard-google-ai-search-updates/>, accessed on 2024-04-06.
- [8] "A statistical test suite for random and pseudorandom number generators for cryptographic applications (revision 1a)," April 2010, accessed on 2024-04-06. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-22r1a.pdf>
- [9] GeeksforGeeks, "Quick sort in c," <https://www.geeksforgeeks.org/quick-sort-in-c/>, accessed on 2024-04-06.
- [10] K. [kokke], "tiny-aes-c," <https://github.com/kokke/tiny-AES-c>, accessed on 2024-04-06.
- [11] HLSLibs, "HLSLibs - high-level synthesis libraries," <https://hlslibs.org/>, accessed on 2024-04-06.

1	Original c code (after removing printf s)
1	<code>void Frequency(int n)</code>
2	<code>{</code>
3	<code> int i;</code>
4	<code> double f, s_obs, p_value, sum, sqrt2 =</code>
5	<code> 1.41421356237309504880;</code>
6	<code> sum = 0.0;</code>
7	<code> for (i=0; i<n; i++)</code>
8	<code> sum += 2*(int)epsilon[i]-1;</code>
9	<code> s_obs = fabs(sum)/sqrt(n);</code>
10	<code> f = s_obs/sqrt2;</code>
11	<code> p_value = erfc(f);</code>
11	<code>}</code>
1	HLS compatible code rewritten by the LLM
1	<code>#define TARGET_BITS 128</code>
2	<code>#define P_VALUE_THRESHOLD 0.01</code>
3	<code>#define THRESHOLD 29</code>
4	<code>typedef ac_int<9, true> sum_type; // true for</code>
5	<code> signed</code>
6	<code>typedef ac_int<8, false> count_type; // false</code>
7	<code> for unsigned</code>
8	<code>typedef ac_int<1, false> bit_type;</code>
9	<code>void</code>
10	<code>Frequency_Bit(bit_type *bit, bit_type *valid,</code>
11	<code> bit_type *result)</code>
12	<code>{</code>
13	<code> static sum_type sum = 0;</code>
14	<code> static count_type count = 0;</code>
15	<code> sum += (*bit) ? sum_type(1) : sum_type(-1);</code>
16	<code> count++;</code>
17	<code> if (count == TARGET_BITS) {</code>
18	<code> // Check if sum is within the threshold</code>
19	<code> range</code>
20	<code> *result = (sum <= THRESHOLD) && (sum >= -</code>
21	<code> THRESHOLD);</code>
22	<code> *valid = 1;</code>
23	<code> count = 0;</code>
24	<code> sum = 0;</code>
25	<code> } else {</code>
26	<code> *result = 0;</code>
27	<code> *valid = 0;</code>
28	<code> }}</code>

Fig. 6: Human-in-the-loop Monobit

APPENDIX

A. LLM generated HLS compatible C codes

In this section we report original codes and final codes of our case study. We cut print statements from the originals and comments from the results.

B. Waveforms

C. Conversations

Here we list the links to the conversations to go from C to HLS C.

- Frequency Test: <https://g.co/gemini/share/ba393c5de5a6>
- Frequency Block Test: <https://g.co/gemini/share/659379d677c0>
- Cumulative Sums Test: <https://g.co/gemini/share/0f35a4d248e7>
- QuickSort: <https://g.co/gemini/share/92b68e7849fc>
- AES 128: <https://g.co/gemini/share/92b68e7849fc>

```

1 Original c code (after removing printf s)
2
3 void BlockFrequency(int M, int n)
4 {
5     int i, j, N, blockSum;
6     double p_value, sum, pi, v,
7         chi_squared;
8     N = n/M; /* # OF
9         SUBSTRING BLOCKS */
10    sum = 0.0;
11
12    for ( i=0; i<N; i++ ) {
13        blockSum = 0;
14        for ( j=0; j<M; j++ )
15            blockSum += epsilon[j+i
16                *M];
17        pi = (double)blockSum/(double)M
18            ;
19        v = pi - 0.5;
20        sum += v*v;
21    }
22    chi_squared = 4.0 * M * sum;
23    p_value = cephes_igamc(N/2.0,
24        chi_squared/2.0);
25 }
26
27 HLS compatible code rewritten by the LLM
28
29 void BlockFrequency(ac_int<1, false> bit,
30     ac_int<1, false>& valid, ac_int<1, false>&
31     result){
32     static ac_int<7, false> blockSum = 0;
33     static ac_fixed<16, 8, false> sum = 0.0;
34     ac_fixed<8, 4, false> pi_fixed, v;
35     static ac_int<4, false> i = 0;
36     static ac_int<8, false> j = 0;
37     if (j == 0) {
38         i++;
39         blockSum = 0;
40     }
41     blockSum += bit;
42     j++;
43     if (j == BLOCK_SIZE) {
44         pi_fixed = (ac_fixed<12, 4, false>)
45             blockSum / BLOCK_SIZE;
46         const ac_fixed<8, 4, false> half = 0.5;
47         v = pi_fixed - half;
48         sum += v * v;
49         j = 0;
50     }
51     if (i == NUM_BLOCKS) {
52         const ac_fixed<16, 8, false> threshold =
53             13.8155 / 512.0;
54         valid = true;
55         result = (sum < threshold) ? 0 : 1;
56     } else {
57         valid = false;
58         result = 0;
59     }
60 }

```

Fig. 7: Human-in-the-loop Monobit Block

- Overlapping Template Matching Test:
<https://g.co/gemini/share/c1ac0cef56f2>

```

1 Original c code (after removing printf s)
2
3 void CumulativeSums(int n) {
4     int S, sup, inf, z, zrev, k;
5     double sum1, sum2, p_value;
6     S = 0; sup = 0; inf = 0;
7     for (k = 0; k < n; k++) {
8         epsilon[k] ? S++ : S--;
9         if (S > sup) sup++;
10        if (S < inf) inf--;
11        z = (sup > -inf) ? sup : -inf;
12        zrev = (sup - S > S - inf) ? sup - S :
13            S - inf;
14        sum1 = 0.0; // forward
15        for (k = (-n / z + 1) / 4; k <= (n / z - 1)
16            / 4; k++) {
17            sum1 += cephes_normal(((4 * k + 1) * z)
18                / sqrt(n));
19            sum1 -= cephes_normal(((4 * k - 1) * z)
20                / sqrt(n));
21        }
22        sum2 = 0.0;
23        for (k = (-n / z - 3) / 4; k <= (n / z - 1)
24            / 4; k++) {
25            sum2 += cephes_normal(((4 * k + 3) * z)
26                / sqrt(n));
27            sum2 -= cephes_normal(((4 * k + 1) * z)
28                / sqrt(n));
29        }
30        p_value = 1.0 - sum1 + sum2;
31        sum1 = 0.0; // backwards
32        for (k = (-n / zrev + 1) / 4; k <= (n /
33            zrev - 1) / 4; k++) {
34            sum1 += cephes_normal(((4 * k + 1) *
35                zrev) / sqrt(n));
36            sum1 -= cephes_normal(((4 * k - 1) *
37                zrev) / sqrt(n));
38        }
39        sum2 = 0.0;
40        for (k = (-n / zrev - 3) / 4; k <= (n /
41            zrev - 1) / 4; k++) {
42            sum2 += cephes_normal(((4 * k + 3) *
43                zrev) / sqrt(n));
44            sum2 -= cephes_normal(((4 * k + 1) *
45                zrev) / sqrt(n));
46        }
47        p_value = 1.0 - sum1 + sum2;
48    }
49 }
50
51 HLS compatible code rewritten by the LLM
52
53 #define N 20000
54 #define THRESHOLD (ac_int<12, false>)397
55 void CumulativeSums_Streaming(ac_int<1, false>
56     bit, ac_int<1, false>* output, ac_int<1,
57     false>* valid) {
58     static ac_int<12, false> S = 0;
59     static ac_int<12, false> sup = 0;
60     static ac_int<12, false> inf = 0;
61     S += bit ? 1 : -1;
62     sup = sup > S ? sup : S;
63     inf = inf < S ? inf : S;
64     // Logic for output based on sup and inf
65     static ac_int<16, false> processed_bits = 0;
66     processed_bits++;
67     if (processed_bits == N) {
68         *output = (sup < THRESHOLD && inf > -
69             THRESHOLD) ? 1 : 0;
70         *valid = (processed_bits == N);
71         processed_bits = 0;
72     } else {
73         *output = 0;
74         *valid = 0;
75     }
76 }

```

Fig. 8: Human-in-the-loop Cusums

```

1 Original c code (after removing printf s)
2
3 void OverlappingTemplateMatchings(int m, int n)
4 {
5     int i, k, match;
6     double W_obs, eta, sum, chi2,
7         p_value, lambda;
8     int M, N, j, K = 5;
9     unsigned int nu[6] = { 0, 0, 0, 0, 0, 0 };
10    //double pi[6] = {
11        0.143783, 0.139430, 0.137319, 0.124314,
12        0.106209, 0.348945 };
13    double pi[6] = { 0.364091,
14        0.185659, 0.139381, 0.100571, 0.0704323,
15        0.139865 };
16    BitSequence *sequence;
17    M = 1032;
18    N = n/M;
19    if ( (sequence = (BitSequence *) calloc(m,
20        sizeof(BitSequence))) == NULL ) {
21        // ERROR
22    }
23    else
24        for ( i=0; i<m; i++ )
25            sequence[i] = 1;
26    lambda = (double) (M-m+1)/pow(2,m);
27    eta = lambda/2.0;
28    sum = 0.0;
29    for ( i=0; i<K; i++ ) {
30        /* Compute Probabilities */
31        pi[i] = Pr(i, eta);
32        sum += pi[i];
33    }
34    pi[K] = 1 - sum;
35    for ( i=0; i<N; i++ ) {
36        W_obs = 0;
37        for ( j=0; j<M-m+1; j++ ) {
38            match = 1;
39            for ( k=0; k<m; k++ ) {
40                if ( sequence[k] != epsilon[i*M+j+k] )
41                    match = 0;
42            }
43            if ( match == 1 )
44                W_obs++;
45        }
46        if ( W_obs <= 4 )
47            nu[(int)W_obs]++;
48        else
49            nu[K]++;
50    }
51    sum = 0;
52    chi2 = 0.0;
53    /* Compute Chi Square */
54    for ( i=0; i<K+1; i++ ) {
55        chi2 += pow((double)nu[i] - (double)N*pi[i],
56            2)/((double)N*pi[i]);
57        sum += nu[i];
58    }
59    p_value = cephes_igamc(K/2.0, chi2/2.0);
60 }

```

Fig. 9: Overlapping Original

```

1 HLS compatible code rewritten by the LLM
2
3 #define N 1048576
4 #define M 1032
5 #define N_WIDTH 20
6 #define M_WIDTH 10
7 #define CHI2_THRESHOLD_FIXED ac_fixed<21, 21,
8     false>(1056300.0)
9 void OverlappingTemplateMatchings(int epsilon,
10     ac_int<1, false> *is_random_out, ac_int<1,
11     false> *valid_output_out) {
12     static ac_int<9, false> W_obs = 0;
13     static ac_int<10, false> nu[6] = {0, 0, 0, 0,
14         0, 0};
15     static ac_int<9, false> shift_reg[9] = {0};
16     static ac_int<N_WIDTH+1, false> n_counter =
17         0;
18     static const ac_fixed<32, 16, false> pi[6] =
19         {
20             2.746566,
21             ac_fixed<32, 16, false>(5.386218), ac_fixed
22             <32, 16, false>(7.17457),
23             ac_fixed<32, 16, false>(9.94322), ac_fixed
24             <32, 16, false>(14.198031), ac_fixed
25             <32, 16, false>(7.1497515)
26         };
27     ac_fixed<23, 22> chi2 = 0;
28     for (int i = 8; i > 0; i--) {
29         shift_reg[i] = shift_reg[i - 1];
30     }
31     shift_reg[0] = ac_int<1, false>(epsilon);
32     ac_int<1, false> match = 1;
33     for (int i = 0; i < 9; i++) {
34         if (shift_reg[i] != 1) {
35             match = 0;
36             break;
37         }
38     }
39     if (match) {
40         W_obs++;
41     }
42     n_counter++;
43     if (n_counter % M == M-1) {
44         if (W_obs <= 4) {
45             nu[(int)W_obs]++;
46         } else {
47             nu[5]++;
48         }
49         W_obs = 0;
50     }
51     *is_random_out = 0;
52     *valid_output_out = 0;
53     if (n_counter == N) {
54         for (int i = 0; i < 6; i++) {
55             chi2 += nu[i] * nu[i] * pi[i];
56         }
57         *is_random_out = chi2 <
58             CHI2_THRESHOLD_FIXED;
59         *valid_output_out = true;
60         for (int i = 0; i < 6; i++) {
61             nu[i] = 0;
62         }
63         n_counter = 0;
64     }
65 }

```

Fig. 10: Human-in-the-loop Overlapping

```

1 Original c code (after removing printf s)
1 void swap(int* a, int* b) {
2     int temp = *a;
3     *a = *b;
4     *b = temp; }
5 int partition(int arr[], int low, int high) {
6     int pivot = arr[low];
7     int i = low;
8     int j = high;
9     while (i < j) {
10         while (arr[i] <= pivot && i <= high -
11             1) {
12             i++;
13         }
14         while (arr[j] > pivot && j >= low + 1)
15             j--;
16         if (i < j) {
17             swap(&arr[i], &arr[j]);
18         }
19     }
20     swap(&arr[low], &arr[j]);
21     return j; }
22 void quickSort(int arr[], int low, int high) {
23     if (low < high) {
24         int partitionIndex = partition(arr, low
25             , high);
26         quickSort(arr, low, partitionIndex - 1)
27         ;
28         quickSort(arr, partitionIndex + 1, high
29             );
30     } }

```

Fig. 11: Quicksort original

```

1 HLS compatible code rewritten by the LLM
1 int partition(int arr[MAX_SIZE], int low, int
2     high) {}
3     int pivot = arr[low];
4     int i = low;
5     int j = high;
6     if (low == high) {
7         return low;
8     }
9     while (i < j) {
10         while (arr[i] <= pivot && i <= high -
11             1) {
12             i++;
13         }
14         while (arr[j] > pivot && j >= low + 1)
15             j--;
16         if (i < j) {
17             int temp = arr[i];
18             arr[i] = arr[j];
19             arr[j] = temp;
20         }
21     }
22     int temp = arr[low];
23     arr[low] = arr[j];
24     arr[j] = temp;
25     return j;
26 void quickSort(int arr[MAX_SIZE], int low, int
27     high) {
28     int stack[100];
29     int top = -1;
30     if (high - low + 1 > MAX_SIZE) {
31         return;
32     }
33     stack[++top] = low;
34     stack[++top] = high;
35     while (top >= 0) {
36         high = stack[top--];
37         low = stack[top--];
38         int partitionIndex = partition(arr, low,
39             high);
40         if (partitionIndex + 1 < high) {
41             stack[++top] = partitionIndex + 1;
42             stack[++top] = high;
43         }
44         if (low < partitionIndex - 1) {
45             stack[++top] = low;
46             stack[++top] = partitionIndex - 1;
47         }
48     } }

```

Fig. 12: Human-in-the-loop Quicksort

1	Original c code (after removing printf s)
1	<code>static void Cipher(state_t* state, const uint8_t* RoundKey)</code>
2	{
3	uint8_t round = 0;
4	AddRoundKey(0, state, RoundKey);
5	for (round = 1; ; ++round)
6	{
7	SubBytes(state);
8	ShiftRows(state);
9	if (round == Nr) {
10	break;
11	}
12	MixColumns(state);
13	AddRoundKey(round, state, RoundKey);
14	}
15	AddRoundKey(Nr, state, RoundKey);
16	}
1	HLS compatible code rewritten by the LLM
1	<code>void Cipher(state_t state[4][4], const uint8_t RoundKey[AES_keyExpSize]) {</code>
2	uint8_t round;
3	AddRoundKey(0, state, RoundKey);
4	for (round = 1; round <= Nr; ++round) {
5	SubBytes(state);
6	ShiftRows(state);
7	if (round < Nr) {
8	MixColumns(state);
9	}
10	AddRoundKey(round, state, RoundKey);
11	}
12	}

Fig. 13: Human-in-the-loop AES Cipher

1	Original c code (after removing printf s)
1	<code>static void MixColumns(state_t* state)</code>
2	{
3	uint8_t i;
4	uint8_t Tmp, Tm, t;
5	for (i = 0; i < 4; ++i)
6	{
7	t = (*state)[i][0];
8	Tmp = (*state)[i][0] ^ (*state)[i][1] ^ (*state)[i][2] ^ (*state)[i][3];
9	Tm = (*state)[i][0] ^ (*state)[i][1];
10	Tm = xtime(Tm); (*state)[i][0] ^= Tm ^ Tmp;
11	Tm = (*state)[i][1] ^ (*state)[i][2];
12	Tm = xtime(Tm); (*state)[i][1] ^= Tm ^ Tmp;
13	Tm = (*state)[i][2] ^ (*state)[i][3];
14	Tm = xtime(Tm); (*state)[i][2] ^= Tm ^ Tmp;
15	Tm = (*state)[i][3] ^ t;
16	Tm = xtime(Tm); (*state)[i][3] ^= Tm ^ Tmp;
17	}
18	}
1	HLS compatible code rewritten by the LLM
1	<code>static void MixColumns(state_t state[4][4]) {</code>
2	uint8_t i, Tmp, Tm, t;
3	for (i = 0; i < 4; ++i) {
4	t = state[i][0];
5	Tmp = state[i][0] ^ state[i][1] ^ state[i][2] ^ state[i][3];
6	Tm = state[i][0] ^ state[i][1]; Tm = xtime(Tm);
7	state[i][0] ^= Tm ^ Tmp;
8	
9	// Corrected lines with direct array access
10	Tm = state[i][1] ^ state[i][2]; Tm = xtime(Tm);
11	state[i][1] ^= Tm ^ Tmp;
12	Tm = state[i][2] ^ state[i][3]; Tm = xtime(Tm);
13	state[i][2] ^= Tm ^ Tmp;
14	Tm = state[i][3] ^ t; Tm = xtime(Tm);
15	state[i][3] ^= Tm ^ Tmp;
16	}
17	}

Fig. 14: Human-in-the-loop AES Mix Columns

```

1 Original c code (after removing printf s)
2
3 static void ShiftRows(state_t* state)
4 {
5     uint8_t temp;
6
7     // Rotate first row 1 columns to left
8     temp = (*state)[0][1];
9     (*state)[0][1] = (*state)[1][1];
10    (*state)[1][1] = (*state)[2][1];
11    (*state)[2][1] = (*state)[3][1];
12    (*state)[3][1] = temp;
13
14    // Rotate second row 2 columns to left
15    temp = (*state)[0][2];
16    (*state)[0][2] = (*state)[2][2];
17    (*state)[2][2] = temp;
18
19    temp = (*state)[1][2];
20    (*state)[1][2] = (*state)[3][2];
21    (*state)[3][2] = temp;
22
23    // Rotate third row 3 columns to left
24    temp = (*state)[0][3];
25    (*state)[0][3] = (*state)[3][3];
26    (*state)[3][3] = (*state)[2][3];
27    (*state)[2][3] = (*state)[1][3];
28    (*state)[1][3] = temp;
29 }
30
31 HLS compatible code rewritten by the LLM
32
33 static void ShiftRows(state_t state[4][4]) {
34     uint8_t temp;
35     int i, j;
36
37     // Loop over rows 1 to 3 (0-based indexing)
38     for (i = 1; i < 4; ++i) {
39         // Number of positions to left shift for
40         // current row
41         int shift = i;
42
43         // Use a separate loop for the number of
44         // shifts
45         for (j = 0; j < shift; ++j) {
46             temp = state[0][i]; // Store the element
47             // to be shifted
48             // Shift all elements by one position to
49             // the left
50             for (int k = 0; k < 3; ++k) {
51                 state[k][i] = state[k + 1][i];
52             }
53             state[3][i] = temp; // Place the temp
54             // value at the end
55         }
56     }
57 }

```

Fig. 15: Human-in-the-loop AES Shift Rows

```

1 Original c code (after removing printf s)
2
3 static void SubBytes(state_t* state)
4 {
5     uint8_t i, j;
6     for (i = 0; i < 4; ++i)
7     {
8         for (j = 0; j < 4; ++j)
9         {
10             (*state)[j][i] = getSBoxValue((*state)[j][i]);
11         }
12     }
13 }
14
15 HLS compatible code rewritten by the LLM
16
17 static void SubBytes(state_t state[4][4]) {
18     uint8_t i, j;
19     for (i = 0; i < 4; ++i) {
20         for (j = 0; j < 4; ++j) {
21             state[j][i] = getSBoxValue(state[j][i]);
22         }
23     }
24 }

```

Fig. 16: Human-in-the-loop AES Sub Bytes

```

1 Original c code (after removing printf s)
2
3 static void AddRoundKey(uint8_t round, state_t*
4 state, const uint8_t* RoundKey)
5 {
6     uint8_t i, j;
7     for (i = 0; i < 4; ++i)
8     {
9         for (j = 0; j < 4; ++j)
10         {
11             (*state)[i][j] ^= RoundKey[(round * Nb *
12 4) + (i * Nb) + j];
13         }
14     }
15 }
16
17 HLS compatible code rewritten by the LLM
18
19 static void AddRoundKey(uint8_t round, state_t
20 state[4][4], const uint8_t RoundKey[
21 AES_KEYLEN]) {
22     uint8_t i, j;
23     uint8_t base = round * Nb * 4;
24     for (i = 0; i < 4; ++i) {
25         for (j = 0; j < 4; ++j) {
26             state[i][j] ^= RoundKey[base + (i * Nb) +
27 j];
28         }
29     }
30 }

```

Fig. 17: Human-in-the-loop AES Add Round Key

```

1 HLS compatible code from the hands-free flow
1 // Function to swap two elements
2 void swap(int* a, int* b){
3   int temp = *a;
4   *a = *b;
5   *b = temp;}
6 int partition(int arr[], int low, int high){
7   int pivot = arr[low];
8   int i = low;
9   int j = high;
10  while (i < j){
11    while (arr[i] <= pivot && i <= high - 1){
12      i++;
13    }
14    while (arr[j] > pivot && j >= low + 1){
15      j--;
16    }
17    if (i < j){
18      swap(&arr[i], &arr[j]);
19    }
20  }
21  swap(&arr[low], &arr[j]);
22  return j;
23 }
24 void quickSort(int arr[64], int low, int high){
25   int stack[128]; // Fixed size stack to work
        with HLS tools
26   int top = -1;
27   stack[++top] = low;
28   stack[++top] = high;
29   while (top >= 0){
30     high = stack[top--];
31     low = stack[top--];
32     if (low < high){
33       int partitionIndex = partition(arr, low
        , high);
34       stack[++top] = low;
35       stack[++top] = partitionIndex - 1;
36       stack[++top] = partitionIndex + 1;
37       stack[++top] = high;
38   }}}

```

Fig. 18: Hands-free C2HLSC QuickSort

```

1 HLS compatible code from the hands-free flow
1 void AddRoundKey(uint8_t round, state_t state,
        const uint8_t* RoundKey)
2 {
3   uint8_t i, j;
4   uint8_t RoundKey_local[AES_keyExpSize];
5   for (i = 0; i < AES_keyExpSize; ++i)
6   {
7     #pragma HLS PIPELINE II=1
8     RoundKey_local[i] = RoundKey[i];
9   }
10  for (i = 0; i < 4; ++i)
11  {
12    for (j = 0; j < 4; ++j)
13    {
14      #pragma HLS PIPELINE II=1
15      state[i][j] ^= RoundKey_local[(round *
        Nb * 4) + (i * Nb) + j];
16    }
17  }
18 }

```

Fig. 19: Hands-free C2HLSC AES Add Round Key

```

1 HLS compatible code from the hands-free flow
1 void ShiftRows(state_t state) {
2   uint8_t temp;
3
4   // Rotate first row 1 column to the left
5   temp = state[0][1];
6   state[0][1] = state[1][1];
7   state[1][1] = state[2][1];
8   state[2][1] = state[3][1];
9   state[3][1] = temp;
10
11  // Rotate second row 2 columns to the left
12  temp = state[0][2];
13  state[0][2] = state[2][2];
14  state[2][2] = temp;
15
16  temp = state[1][2];
17  state[1][2] = state[3][2];
18  state[3][2] = temp;
19
20  // Rotate third row 3 columns to the left
21  temp = state[0][3];
22  state[0][3] = state[3][3];
23  state[3][3] = state[2][3];
24  state[2][3] = state[1][3];
25  state[1][3] = temp;
26 }

```

Fig. 20: Hands-free C2HLSC AES Shift Rows

```

1 HLS compatible code from the hands-free flow
1 void MixColumns(state_t state)
2 {
3   uint8_t i;
4   uint8_t Tmp, Tm, t;
5   for (i = 0; i < 4; ++i)
6   {
7     t = state[i][0];
8     Tmp = state[i][0] ^ state[i][1] ^ state
        [i][2] ^ state[i][3];
9     Tm = state[i][0] ^ state[i][1]; Tm =
        xtime(Tm); state[i][0] ^= Tm ^ Tmp;
10    Tm = state[i][1] ^ state[i][2]; Tm =
        xtime(Tm); state[i][1] ^= Tm ^ Tmp;
11    Tm = state[i][2] ^ state[i][3]; Tm =
        xtime(Tm); state[i][2] ^= Tm ^ Tmp;
12    Tm = state[i][3] ^ t; Tm =
        xtime(Tm); state[i][3] ^= Tm ^ Tmp;
13  }
14 }

```

Fig. 21: Hands-free C2HLSC AES Mix Columns

```

1 HLS compatible code from the hands-free flow
1 #define getSBoxValue(num) (sbox[(num)])
2
3 // The SubBytes Function Substitutes the values
  in the
4 // state matrix with values in an S-box.
5 static void SubBytes(state_t state)
6 {
7     uint8_t i, j;
8     for (i = 0; i < 4; ++i)
9     {
10         for (j = 0; j < 4; ++j)
11         {
12             state[j][i] = getSBoxValue(state[j
13             ][i]);
14         }
15     }

```

Fig. 22: Hands-free C2HLSC AES Sub Bytes